Guide d'étude — Modern Data Lakehouse (batch et temps réel)

Nov 13, 2026 · @khadija

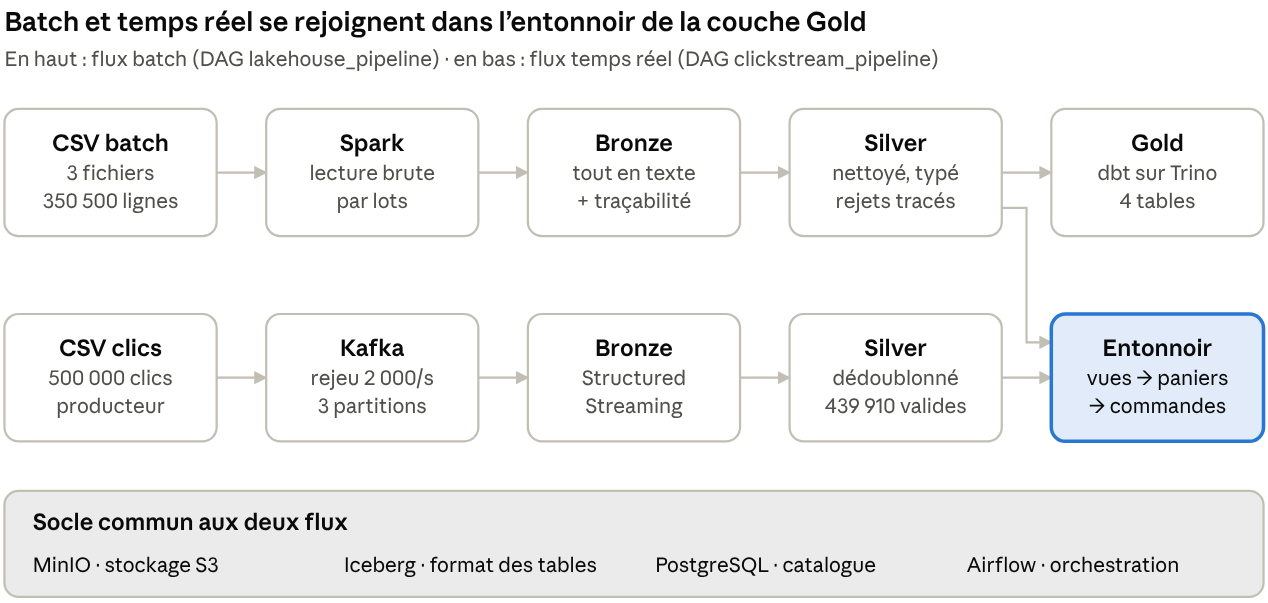
Le pitch en 2 minutes

Ce projet transforme des données e-commerce volontairement sales en tableaux fiables, avec deux modes d'arrivée des données : par lots (batch) et en continu (temps réel). Voici comment le raconter, à l'oral, en 4 temps.

1. **Le problème.** J'ai pris un jeu de données Kaggle « Customer 360 » : 50 000 clients, 500 produits, 300 000 commandes et 500 000 clics de navigation. Les données sont sales exprès : doublons, montants « NaN », dates impossibles, catégories mal écrites. Le but : obtenir des indicateurs fiables sans jamais perdre la trace de la donnée d'origine.
2. **L'architecture.** J'ai construit un lakehouse en 3 couches. Bronze garde la donnée brute, Silver la nettoie, Gold calcule les indicateurs métier. Les tables sont au format Apache Iceberg, stockées dans MinIO (un stockage compatible S3).
3. **Les outils, chacun à sa place.** Spark nettoie les données (Bronze, Silver). dbt calcule la couche Gold en SQL, exécuté par Trino. Kafka et Spark Structured Streaming font arriver les clics en continu. Airflow orchestre le tout, avec un contrôle qualité qui bloque la suite en cas de problème.
4. **Ce que j'en retiens.** Les vrais apprentissages sont les problèmes trouvés : un bug de « NaN » qui faussait le chiffre d'affaires de 821 millions (vu grâce au time travel d'Iceberg), des dates « bidon » répétées des milliers de fois, et 4 286 commandes validées à montant nul, détectées par un test dbt.

Phrase d'accroche : « J'ai construit un lakehouse qui ingère des données en batch et en temps réel, les nettoie avec des règles traçables, et les croise dans un entonnoir de conversion — et surtout, j'ai appris à trouver les pièges cachés dans les données. »

Vue d'ensemble de l'architecture



architecture · 2 flux, 5 étapes chacun, socle commun

Les commandes nettoyées (Silver batch) alimentent aussi l'entonnoir : c'est le seul endroit où les deux flux se croisent, et c'est ce qui justifie d'avoir ajouté Kafka.

Les outils un par un

Chaque outil a un rôle précis : si tu sais dire *pourquoi lui et pas un autre*, tu as gagné la question.

Outil	Son rôle dans le projet	Pourquoi ce choix	Alternative à citer
Apache Spark 3.5 (PySpark)	Lit les CSV, nettoie et joint les données (Bronze, Silver)	Traite de gros volumes en parallèle ; permet des règles complexes en Python (fonctions de normalisation)	pandas (trop limité en volume), Polars, DuckDB

Outil	Son rôle dans le projet	Pourquoi ce choix	Alternative à citer
Apache Iceberg	Format des tables : données en fichiers Parquet + métadonnées	Transactions fiables, time travel (revenir à une version), évolution de schéma, partitionnement caché ; standard ouvert lu par Spark, Trino, Snowflake	Delta Lake, Apache Hudi
MinIO	Stockage des fichiers, un bucket par couche	Compatible avec l'API S3 d'AWS : même code qu'en cloud, mais en local et gratuit	AWS S3, Azure Data Lake Storage
Catalogue JDBC (PostgreSQL)	Annuaire des tables : nom de table → emplacement de ses métadonnées	Lu par Spark **et** par Trino ; réutilise le PostgreSQL déjà présent pour Airflow	Catalogue REST (Polaris, Nessie), Hive Metastore, AWS Glue
Trino	Moteur SQL qui interroge les tables Iceberg	Réponses SQL rapides sans Spark ; moteur d'exécution de dbt	Spark SQL, Dremio, Snowflake
dbt	Construit la couche Gold en SQL, avec des tests	Transformations versionnées, testées et documentées ; lisibles par un analyste	Scripts SQL à la main, PySpark
Apache Airflow	Lance les étapes dans l'ordre (DAG)	Standard du marché ; une étape en échec bloque la suite ; relances automatiques	Dagster, Prefect
Apache Kafka	File d'événements où arrivent les clics	Reçoit un flux continu, le garde sur disque, permet de le relire	RabbitMQ, Redpanda, AWS Kinesis
Spark Structured Streaming	Lit Kafka par petits lots et écrit dans Iceberg	Même outil que le batch ; reprise exacte grâce au checkpoint	Apache Flink, Kafka Connect

Outil	Son rôle dans le projet	Pourquoi ce choix	Alternative à citer
Great Expectations	Suites de règles qualité documentées (rapport HTML)	Formalise et affiche les règles déjà codées	Soda, tests dbt
GitHub Actions (CI)	À chaque envoi : ruff, dbt parse, 30 tests pytest, build Docker	Une erreur est détectée avant d'arriver sur `main`	GitLab CI, Jenkins
Docker Compose	Lance tous les services d'une commande	Environnement identique chez tout le monde	Kubernetes (trop lourd ici)

À retenir sur Iceberg : une table Iceberg n'est pas un fichier, c'est un ensemble de fichiers Parquet plus des métadonnées qui disent quels fichiers forment la table à chaque version (un *snapshot*). C'est ce qui permet le time travel : relire la table telle qu'elle était avant.

Le flux batch, étape par étape

Le DAG Airflow `lakehouse_pipeline` enchaîne 7 tâches ; chacune doit réussir pour que la suivante démarre.

1. **ingest_bronze** (`spark/ingestion/ingest_sources.py`) — lit les 3 CSV et les écrit tels quels dans Bronze. Tout est lu en texte (`string`) : on ne laisse pas Spark deviner un type, sinon une valeur sale serait perdue sans bruit. On ajoute 3 colonnes de traçabilité : `_ingested_at` , `_source_file` , `_batch_id` . Résultat : 350 500 lignes.
2. **bronze_quality_check** — vérifie que les 3 tables existent et ne sont pas vides.
3. **transform_silver** (`spark/silver/silver_jobs.py`) — nettoie chaque table :
   - clients : dédoublonnage sur `customer_id` (50 000 → 48 200), noms et emails normalisés, 1 000 rejetés (email manquant) ;
   - produits : catégorie normalisée (« hom3 » → « home », « ele » → « electronics »), prix converti ; 405 gardés, 95 rejetés ;
   - commandes : statut, moyen de paiement et quantité normalisés, date au format `yyyy-MM-dd` ; 150 137 gardées, 149 863 rejetées ;
   - jointure commandes + clients + produits : 118 909 commandes enrichies ; 31 228 « orphelines » (client ou produit rejeté avant).
4. **silver_quality_check** — contrôle bloquant sur Silver.

5. **dbt_build_gold** — `dbt build` construit les 4 tables Gold avec Trino et lance les tests. Seules les commandes au statut `success` comptent dans le chiffre d'affaires (87 627 commandes).
6. **gold_quality_check** — les 4 tables existent, et le CA total est un nombre positif fini (protection contre le bug NaN).
7. **generate_report** — écrit un rapport Markdown avec les chiffres de chaque couche.

Chaque ligne rejetée garde son motif. La colonne `_rejection_reason` liste les champs fautifs, par exemple `order_date, quantity`. On peut donc expliquer le taux de rejet : sur 300 000 commandes, 90 000 ont une date invalide, 44 867 une quantité invalide, 38 537 un montant invalide, 10 586 un moyen de paiement inconnu (une ligne peut cumuler plusieurs motifs).

Les 4 tables Gold

Table	Grain (une ligne =)	Lignes	Ce qu'elle répond
`sales_daily`	un jour	730	Combien vend-on par jour ? panier moyen ?
`sales_by_product`	un produit	405	Quels produits rapportent le plus ?
`sales_by_region`	un pays	243	Où vend-on ?
`customer_metrics`	un client	39 370	Combien dépense chaque client ?

Au total, 43,6 % des lignes sources sont écartées en Silver (152 758 sur 350 500). C'est voulu : le jeu de données est construit pour être sale.

Le flux temps réel (Kafka), étape par étape

Les clics arrivent en continu, pas en lots : c'est un besoin différent du batch, d'où Kafka. Et c'est seulement en croisant clics et commandes qu'on obtient l'entonnoir de conversion.

1. **Le producteur** (`streaming/producer/replay_clickstream.py`) relit le CSV de 500 000 clics et envoie chaque ligne comme un message JSON dans le *topic* Kafka `clickstream`, à 2 000 messages par seconde par défaut. **C'est une simulation** : dis-le toujours. La clé de chaque message est `event_id`. Le producteur n'envoie rien en double côté Kafka (option `enable.idempotence`).

2. **Kafka** range les messages dans 3 *partitions* et les garde sur disque. Un lecteur peut donc les relire, ou reprendre là où il s'était arrêté grâce à l'*offset* (le numéro du message dans sa partition).
3. **Spark Structured Streaming** (`spark/streaming/stream_clickstream_bronze.py`) lit Kafka par *micro-lots* (100 000 messages au plus) et ajoute chaque lot à la table `bronze.ecommerce.clickstream`. Il note dans un *checkpoint* (stocké dans MinIO) les offsets déjà traités. Deux modes :
   - `available-now` : traite tout ce qui attend, puis s'arrête (utilisé par Airflow) ;
   - `continuous` : un micro-lot toutes les 10 secondes, sans s'arrêter.
4. **Silver** (`spark/silver/silver_clickstream.py`) :
   - dédoublonne sur `event_id` (un message renvoyé deux fois ne compte qu'une fois) ;
   - accepte 4 écritures valides de la date, rejette les dates vides (20 160), la date impossible `2024/31/01 25:61:00` (20 011) et les dates postérieures à l'extraction du fichier (19 919) ;
   - extrait le `product_id` de l'adresse de la page (`/product/PROD-0109`). Résultat : 439 910 clics valides, du 2 septembre au 2 décembre 2025.
5. **Gold** (`dbt/models/gold/product_funnel.sql`) — par produit, sur les 90 jours communs aux deux sources : 201 481 vues → 50 054 ajouts au panier (24,8 %) → 10 950 commandes (21,9 % des paniers, soit 5,4 % des vues).

« Exactly-once », expliqué simplement. Si le job s'arrête au milieu, il ne perd rien et ne compte rien deux fois. Deux mécanismes s'y combinent : le checkpoint retient où reprendre dans Kafka, et Iceberg enregistre chaque micro-lot en une seule transaction (tout ou rien). Vérifié : une relance ajoute 0 ligne ; 5 000 messages renvoyés ajoutent exactement 5 000 lignes en Bronze, retirées ensuite par le dédoublonnage Silver.

Kafka est optionnel. Il ne démarre qu'avec `docker compose --profile streaming up`, et le streaming a son propre DAG (`clickstream_pipeline`). Sans lui, la partie batch fonctionne seule.

Les découvertes et décisions (tes histoires d'entretien)

Un recruteur retient une histoire « problème → comment je l'ai vu → ce que j'ai fait → résultat » bien plus qu'une liste d'outils. En voici 7, toutes vraies dans ton code.

1. Le NaN invisible (821 millions). La chaîne « NaN » convertie en nombre donne un nombre spécial *NaN*, pas une valeur vide : le test `isNull()` ne le voit pas. Des montants NaN passaient donc en Gold et faussaient le chiffre d'affaires. En comparant deux versions de `sales_daily` avec le time travel d'Iceberg, j'ai mesuré un écart de 821 millions. Correction : un test explicite `isnan()`. Depuis, `check_gold.py` et un test dbt bloquent tout CA non fini.

2. Les dates « bidon » des commandes. 90 000 commandes sont rejetées pour leur date. J'ai d'abord cru qu'on pouvait les récupérer en lisant d'autres formats. En vérifiant, chaque format « alternatif » ne contient qu'**une seule valeur répétée** : `2024/31/01` 18 110 fois, `31-12-2023` 18 050 fois. Ce sont des valeurs sentinelles, pas de vraies dates. Les convertir aurait créé trois pics artificiels dans les ventes. Leçon : regarder la distribution des valeurs avant de « réparer ».

3. Les commandes à montant 0. Un test dbt (`positive`) a échoué : 693 clients avaient dépensé 0. Cause : 12 988 commandes sources ont « 0 » comme montant, et la règle Silver accepte `montant >= 0`. Résultat : 4 286 commandes réussies sans revenu. J'ai gardé le comportement, mais ajouté un avertissement qui les signale à chaque exécution. **Décision encore ouverte** : les rejeter (tous les chiffres changent) ou les garder.

4. Pourquoi dbt n'avait pas marché au début. dbt avec Spark (`dbt-spark`) ne gérait qu'un seul catalogue, et le projet en a trois. Trino, lui, en lit plusieurs, mais pas le type « hadoop » utilisé au départ (sa documentation liste : hive_metastore, glue, jdbc, rest, nessie, snowflake). Solution : passer le catalogue en JDBC (PostgreSQL), lu par Spark et Trino. Les tables existantes ont été migrées sans réécrire une seule donnée (`register_table`), historique conservé.

5. Un centime d'écart entre Spark et Trino. En comparant la Gold dbt à l'ancienne Gold Spark, 370 paniers moyens différaient d'un centime. Cause : un nombre comme 12,345 est stocké en binaire 12,34499…, et les deux moteurs n'arrondissent pas pareil. Correction : une macro dbt qui arrondit en décimal. Résultat : **0 valeur différente** sur les 4 tables.

6. Une date de référence fixe. Pour rejeter les clics « dans le futur », je compare à la date d'extraction du fichier (2 décembre 2025, 16h05), pas à la date du jour. Sinon le résultat changerait selon le jour d'exécution, un bug déjà rencontré sur les commandes. Un pipeline doit donner le même résultat à chaque exécution (*reproductibilité*).

7. La CI et Java 17. Les tests Spark ont échoué sur GitHub alors qu'ils passaient en local : la machine de GitHub était passée à un Java trop récent pour Spark 3.5 (qui supporte Java 8, 11 et 17). Correction : fixer Java 17 dans la CI plutôt que dépendre de la version par défaut.

Questions d'entretien probables et réponses modèles

Réponds toujours en 3 temps : la réponse courte, le pourquoi, un exemple tiré du projet. Entraîne-toi à voix haute, sans lire.

Q1. Pourquoi une architecture en 3 couches (médaillon) ? Pour séparer les responsabilités. Bronze garde la donnée brute pour pouvoir tout rejouer, Silver la rend fiable, Gold répond aux questions métier. Exemple : quand j'ai découvert le bug NaN, j'ai corrigé Silver et tout recalculé depuis Bronze, sans retoucher les sources.

Q2. Pourquoi tout lire en texte dans Bronze ? Si Spark devine un type, une valeur qui ne colle pas devient vide sans alerte. En texte, rien n'est perdu ; le typage se fait en Silver, où chaque échec devient un rejet tracé.

Q3. Iceberg, ça apporte quoi par rapport à de simples fichiers Parquet ? Des transactions (une écriture réussit entièrement ou pas du tout), le time travel, l'évolution de schéma sans tout réécrire, et le partitionnement caché. Exemple : le time travel m'a permis de mesurer l'écart de 821 millions.

Q4. À quoi sert un catalogue ? Pourquoi JDBC ? Le catalogue dit où se trouvent les métadonnées de chaque table. Il faut que Spark et Trino lisent le même. Trino ne lit pas le type « hadoop », d'où JDBC, qui réutilise le PostgreSQL déjà là. En production, je regarderais un catalogue REST (Polaris, Nessie).

Q5. Pourquoi dbt pour Gold, alors que Spark marchait ? Gold, ce sont des agrégats : du SQL pur. dbt les rend lisibles, testés et documentés. Preuve concrète : un test dbt a trouvé les 4 286 commandes à montant 0. Et j'ai vérifié que la nouvelle Gold donnait exactement les mêmes résultats que l'ancienne.

Q6. Ton streaming est-il vraiment en temps réel ? C'est une simulation : un programme rejoue un fichier dans Kafka. Mais toute la chaîne en aval est réelle (Kafka, micro-lots, checkpoint, reprise). Brancher une vraie source ne changerait que le producteur.

Q7. Que se passe-t-il si le job de streaming plante ? Il redémarre depuis le dernier checkpoint. Les offsets déjà traités ne sont pas relus, et un micro-lot à moitié écrit n'existe pas pour Iceberg. Testé : relance = 0 ligne en double.

Q8. Et si le producteur envoie deux fois le même événement ? Kafka le garde deux fois ; Bronze aussi, volontairement (on garde tout). Silver dédoublonne sur `event_id`. Testé avec 5 000 doublons volontaires : 505 000 en Bronze, 500 000 en Silver.

Q9. Comment garantis-tu la qualité ? À trois niveaux : des contrôles bloquants dans Airflow après chaque couche ; des motifs de rejet tracés ligne par ligne ; des tests dbt sur Gold, dont une réconciliation du CA entre les 4 tables. Plus 30 tests automatiques dans la CI.

Q10. Qu'améliorerais-tu ? Un chargement incrémental (MERGE) au lieu de tout recalculer ; un catalogue REST ; trancher le sort des montants à 0 avec le métier ; un déploiement sur le cloud avec Terraform. Montrer qu'on connaît ses limites est un point fort.

Limites honnêtes à assumer

Ces points peuvent sortir en entretien ; mieux vaut les dire toi-même, calmement, avec ce que tu ferais ensuite.

- **Données synthétiques.** Les taux de l'entonnoir sont presque identiques d'une catégorie à l'autre, et une « session » mélange environ 43 clients. L'entonnoir montre la mécanique, pas un vrai comportement d'achat.
- **Le chiffre d'affaires est irréaliste** (13,4 milliards, panier moyen d'environ 153 000). Le revenu est calculé comme `montant × quantité` ; si le montant est déjà un total, le CA est surestimé. Ne mets pas ce chiffre en avant.
- **Streaming simulé** : rejeu d'un fichier, pas une vraie source en direct.
- **Tout est recalculé à chaque exécution batch** (`createOrReplace`), pas de chargement incrémental.
- **Une seule machine** : Spark en mode local, Kafka avec un seul broker. Pas de haute disponibilité.
- **Le docker-compose complet n'a pas encore été lancé de bout en bout sur ton PC.** Chaque brique a été testée (vrais Kafka, Trino, Iceberg, PostgreSQL), mais à faire avant d'en parler comme d'une démo.
- **Montants à 0 : décision ouverte.**

Démontrer le projet

Une démo de 5 minutes vaut mieux que 20 minutes d'explications. Prépare-la une fois sur ton PC.

1. Mettre les 4 CSV Kaggle dans `data/sample/` et créer `docker/.env` (`MINIO_ROOT_USER` , `MINIO_ROOT_PASSWORD`).

2. Lancer : `cd docker` puis `docker compose --profile streaming up -d --build` (sans `--profile streaming` si ton PC manque de mémoire : Kafka demande environ 1 Go de plus).
3. Ouvrir Airflow (http://localhost:8080), lancer le DAG `lakehouse_pipeline`, puis `clickstream_pipeline`.
4. Montrer, dans cet ordre :
   - le DAG Airflow tout vert, et une tâche qui bloque si un contrôle échoue ;
   - MinIO (http://localhost:9001) : les buckets bronze, silver, gold ;
   - Trino : `docker exec -it lakehouse-trino trino`, puis `SELECT * FROM gold.ecommerce.product_funnel ORDER BY views DESC LIMIT 5;` ;
   - un rejet expliqué : `SELECT _rejection_reason, count(*) FROM silver.ecommerce.orders_rejected GROUP BY 1;` ;
   - le flux en direct : lancer le job en mode `continuous` et le producteur, puis recompter la table Bronze toutes les 10 secondes ;
   - le tableau Power BI.
5. Finir par une découverte (le NaN ou les montants à 0) : c'est ce qu'on retient.

Glossaire

Terme	En une phrase
Lakehouse	Stockage de fichiers bon marché (data lake) avec la fiabilité d'un entrepôt de données (tables, transactions, SQL)
Architecture médaillon	Organisation en 3 couches : Bronze (brut), Silver (nettoyé), Gold (métier)
Batch	Traitement par lots, à intervalles (ici, sur déclenchement)
Streaming	Traitement des données au fil de leur arrivée
DAG	Graphe des tâches d'Airflow et de leur ordre ; sans boucle
Snapshot	Une version d'une table Iceberg à un instant donné

Terme	En une phrase
Time travel	Relire une table telle qu'elle était à un snapshot passé
Évolution de schéma	Ajouter ou renommer une colonne sans réécrire les données
Partitionnement	Ranger les données par une colonne (ici le mois) pour ne lire que le nécessaire
Catalogue	Annuaire qui donne, pour chaque table, l'emplacement de ses métadonnées
Topic Kafka	File nommée où les producteurs écrivent et les consommateurs lisent
Partition Kafka	Morceau d'un topic, lu en parallèle ; l'ordre est garanti dans une partition
Offset	Numéro d'un message dans sa partition ; sert à reprendre la lecture
Micro-lot	Petit paquet de messages traité d'un coup par Spark Structured Streaming
Checkpoint	Fichier où Spark note ce qu'il a déjà traité, pour reprendre après un arrêt
Exactly-once	Chaque message compte une fois et une seule, même après une panne
Idempotent	Relancer donne le même résultat (pas de doublon)
Valeur sentinelle	Fausse valeur répétée qui remplace une vraie valeur manquante (ex. `2024/31/01`)
NaN	« Not a Number » : nombre spécial, différent d'une valeur vide (`null`)
Modèle dbt	Un fichier SQL qui produit une table ; dbt les lance dans le bon ordre
Test dbt	Requête qui doit renvoyer 0 ligne ; sinon le test échoue

Terme	En une phrase
Entonnoir de conversion	Suite d'étapes (vue → panier → commande) et part qui passe à l'étape suivante
CI	Intégration continue : tests automatiques à chaque envoi de code
Reproductibilité	Même entrée, même résultat, quel que soit le jour d'exécution